EN.530.626: Optimal Control for Space Systems
Homework 4
Due Nov 10th 11:59PM

The code for this assignment can once again be found on GitHub at this link and can be downloaded by running `git clone https://github.com/JHU-ACEL/ocss_hw.git` from a terminal window. Due to a change in the `Dockerfile` to support a mixed-integer programming solver, you will have to rebuild your `Docker` image. Homework submissions and grading will be managed through Canvas.

Problem	**Points Possible**
Problem 1	20
Problem 2	20
Total	**40**

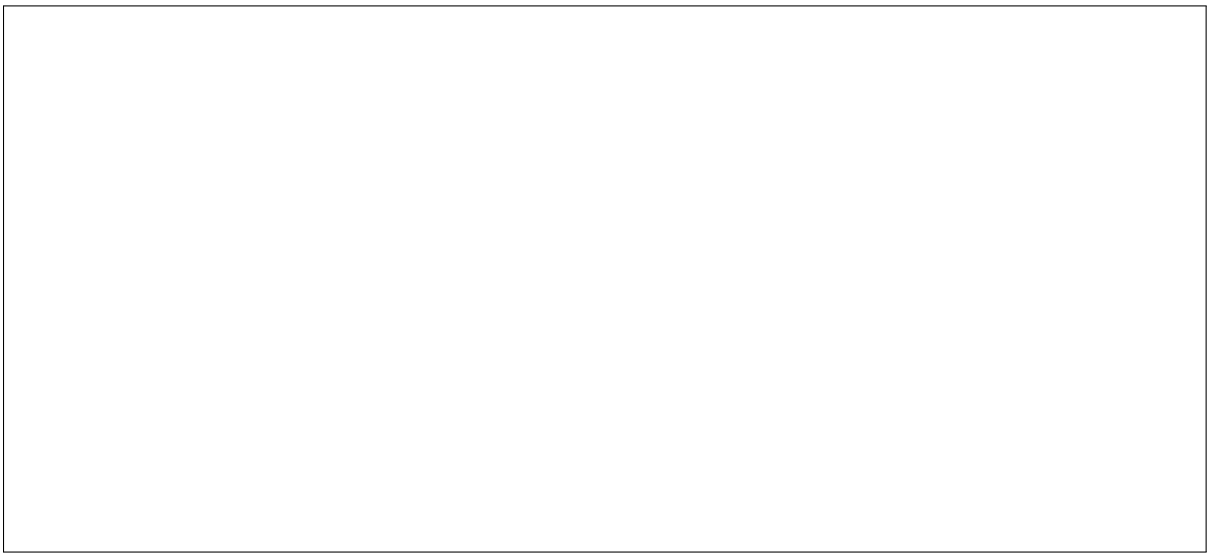
Introduction

This homework will focus on the following technical concepts:

1. Different solution techniques for solving the discrete-time linear quadratic regulator problem.
2. Understanding how derivative-free optimization can be used to solve challenging non-convex problems.

Collaboration:

For each problem in this assignment, indicate in what capacity you collaborated with peers and which peers; if you did not collaborate, then include this information as well. **Your assignment will not be graded without this information.**



Problem 1 [20 Points]: LQR

We consider the discrete-time linear quadratic regulator (LQR) problem,

$$\begin{array}{ll} \underset{x_{0:N}, u_{0:N-1}}{\text{minimize}} & \frac{1}{2} x_N^T Q_g x_N + \sum_{k=0}^{N-1} \frac{1}{2} u_k^T R u_k + x_k^T Q x_k \\ \text{subject to} & x_{k+1} = A x_k + B u_k, \quad k = 0, \ldots, N-1, \\ & x_0 = x_{\text{init}}, \end{array}$$

where $Q_g, Q \succeq 0$ and $R \succ 0$. The state $x_k \in \mathbb{R}^{n_x}$ and control $u_k \in \mathbb{R}^{n_u}$. In this problem, we will show two different ways to compute the LQR solution for some policy $u_k^\star = -K_k x_k$.

Part A [5 Points]

For $N = 2$, write out the linear dynamics constraint as an expression $\overrightarrow{X} = \bar{A} x_{\text{init}} + \bar{B} \overrightarrow{U}$, where $\overrightarrow{X} = (x_0, x_1, x_2)$ and $\overrightarrow{U} = (u_0, u_1)$.



Part B [5 Points]

Plug the expression $\bar{A}x_{\text{init}} + \bar{B}\overrightarrow{U}$ back into the cost function,

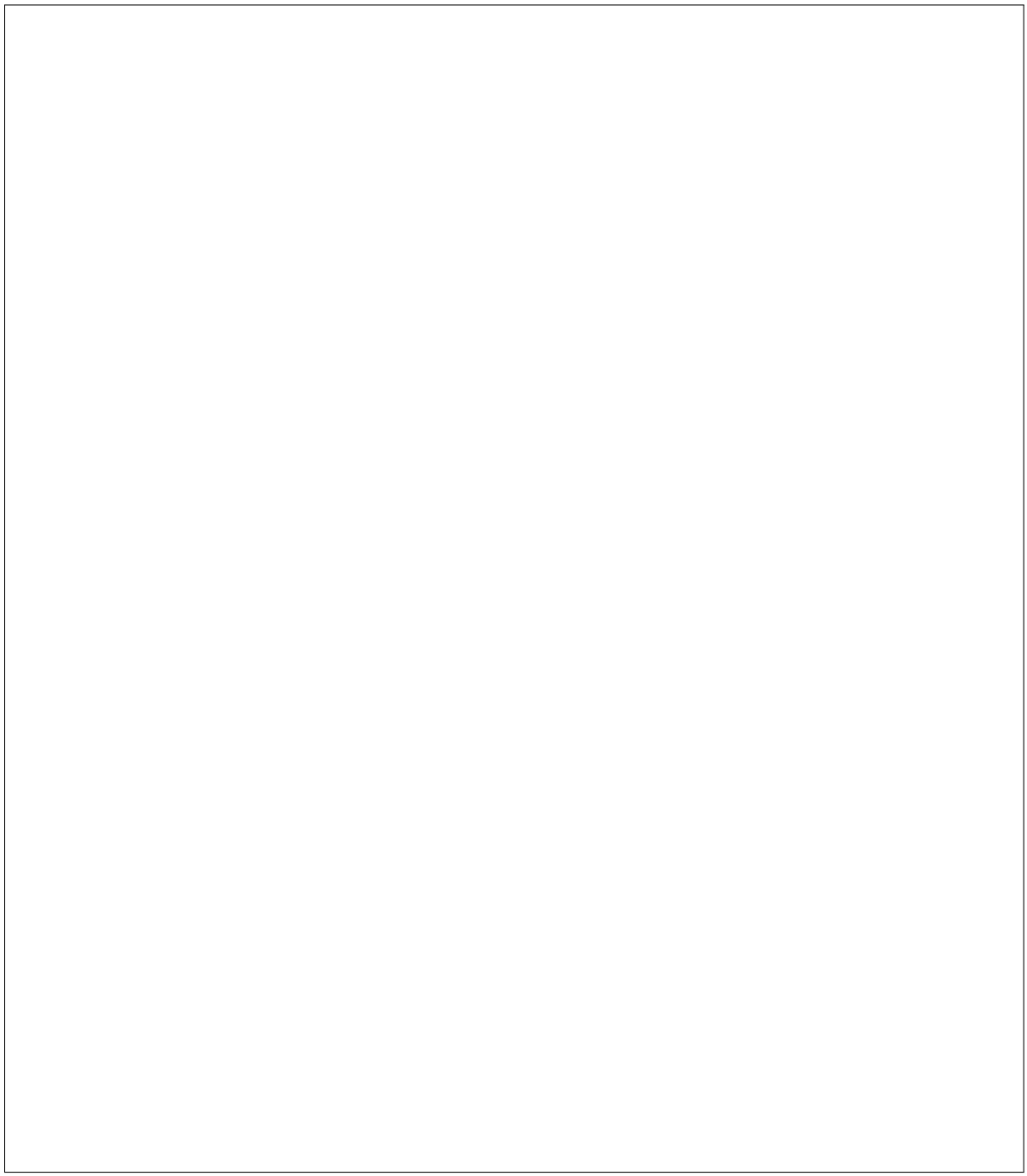
$$J(x_{0:N}, u_{0:N-1}) = \frac{1}{2}x_N^T Q_g x_N + \sum_{k=0}^{N-1} \frac{1}{2}u_k^T R u_k + x_k^T Q x_k$$

and write out a cost term $J(\overrightarrow{U})$ that expresses the cost only in terms of the controls $\overrightarrow{U}$ (i.e., show that the LQR cost is only a function of the control tape by substituting in the linear dynamics constraint). How can

the optimal LQR policy be computed using this expression?

Part C [5 Points]

Instead of using algebraic substitution to write out an expression $J(\overrightarrow{U})$, let us derive the dynamic programming-based solution. Let us assume that the optimal cost-to-go is a quadratic expression, $V_k(z) = \frac{1}{2}z^T P_k z$ and use this to solve for some policy K_k. Compute the LQR gain K_k by applying the necessary conditions of optimality to the expression $V_k(z) = \min_w \frac{1}{2}z^T Q z + \frac{1}{2}w^T R w + V_{k+1}(Az + Bw)$.



Problem 2 [20 Points]: Sampling-Based MPC Implementation

In this problem, we will be implementing MPPI and CEM for a bicycle dynamics model to navigate around an obstacle. Read the instructions provided in `hw4/problem2.py`.

Note that the `MppiCemSolver` class is set up differently than you have seen before. It makes use of the equinox library to make classes compatible with `jit` decorators on functions.

When you are tuning the controller, please submit with your final parameters (e.g. `num_samples`, `num_iter`, `kappa`, `Q`, etc) set as the defaults in the class definition. This is so that when we import to the autograder, we can easily use your tuned controller. We will set `dt`, `T`, and `obstacles` within the autograder. There is no additional write-up required for this problem.

Part A [10 points]

Implement MPPI. Make sure plots are included in marimo notebook pdf.

Part B [10 points]

Implement CEM and run `mujoco` simulation. Attach screenshot of rover at the goal, and the plot produced in `hw4/outputs/` after rover reaches the goal. To run the `mujoco` simulation, just run `start_viewer.sh` in your container. The scripts are already written for you.

Make sure CEM plots and comparison plots are included in marimo notebook pdf submission.

Submission Instruction

- Download the marimo notebook for each problem and join them together into a single PDF or HTML file named `hw4.pdf`.
- Compress the `hw4` folder containing your python files. Name this file "hw4.zip".
- Upload both the PDF/HTML file and the zip folder into the canvas assignment "`Homework 4`".

References